Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 283 (2026) 2243–2250

Procedia Computer Science

www.elsevier.com/locate/procedia

International Conference on Machine Learning and Data Engineering

Integration of Retrieval Augmented Generation (RAG) with Small Language Models (SLMs) for Development of Optimized Agentic AI Chatbot

Pradeep Singh[a], *, N K Nagwani[a], Lilesh Lahre[a]

[a] *Department of Computer Science and engineering, NIT Raipur, Raipur 492001, India*

Abstract

Large Language Models (LLMs) deliver strong performance across natural language tasks but require heavy GPU resources and large memory, making them impractical for smaller devices. Traditional information retrieval (IR) systems, often fails to provide precise contextual grounding for language models, leading to incomplete or misleading responses. This article addresses both issues by integrating Retrieval-Augmented Generation (RAG) with a Small Language Model (SLM), specifically Qwen2.5-7B-Instruct supported by MiniLM-based embeddings. The proposed system constructs a domain-specific knowledge base and retrieves relevant evidence passages to support model responses, thereby ensuring factual accuracy while maintaining a lightweight design. Experiments were conducted on a CPU-only setup (Intel Xeon Silver 4116, 128 GB RAM). The RAG-SLM chatbot achieved a 30% reduction in hallucination rate compared to SLM-only baselines, with an average latency of 8–12 seconds and peak memory usage under 64 GB. These results demonstrate that accurate, transparent, and efficient conversational agents can be deployed without GPU acceleration. The findings highlight the feasibility of agentic chatbots capable of not only retrieving and reasoning over information but also executing domain-specific tasks, making them particularly valuable in enterprise contexts such as insurance claim assistance, customer support automation, healthcare triage, financial advisory, legal document analysis, and technical troubleshooting.

© 2026 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer review under the responsibility of the scientific committee of the International Conference on Machine Learning and Data Engineering

Keywords: Retrieval Augmented Generation (RAG), Large Language Model (LLM), Small Language Models (SLM), Information Retrieval, Natural Language Processing (NLP), Agentic AI, Conversational AI, Low Resource Deployment, Context-Aware Response Generation.

1. Introduction

In recent years, large language models (LLMs) have taken centre stage in natural language processing research and applications. Large language models (LLMs) such as LLaMA and GPT have shown that scaling up the amount of training data and parameters usually improves their performance. These systems are not only capable of producing

1877-0509 © 2026 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer review under the responsibility of the scientific committee of the International Conference on Machine Learning and Data Engineering

text but also handle reasoning tasks and, in some cases, multi-modal inputs that combine different types of data [1]. Due to these abilities, LLMs are now being used in many fields, including healthcare, education, legal research, and information retrieval [2], [3]. The drawback, however, is their heavy demand for resources. Running such models often requires expensive GPUs, very large memory capacity, and specialized infrastructure, making them impractical for individual users or smaller organizations [4], [5]. Even a mid-sized model such as LLaMA-7B can exceed the limits of a normal workstation, while state-of-the-art models with hundreds of billions of parameters are beyond reach for everyday use.

These challenges have shifted attention to small language models (SLMs). Unlike large models, SLMs are designed to be lightweight and can run on ordinary CPUs, laptops, or even mobile devices. Such models provide a useful alternative in situations where cost, energy, or space must be kept under control. The Phi-3 series, for example, has shown that with careful training, compact models can run directly on smartphones while still achieving strong accuracy [6]. Research on compression methods [4] and parameter-efficient tuning [7], [8] has also confirmed that smaller models, when optimized properly, can perform at levels that make them highly practical. For such reason, SLMs are becoming a valuable option for extending access to AI and supporting deployment in everyday computing environments [9]. Lewis et al. [10] first introduced this approach for knowledge-intensive tasks, also Borgeaud et al. [11] demonstrated its effectiveness when retrieving from extremely large text collections. Since then, RAG has been adapted to domains such as biomedical question answering [2], legal reasoning [3], intelligent driving assistance [12], and customer service [13]. These works consistently highlight two advantages: improved factual accuracy and increased user trust in automated systems.

In spite of these advancements, a significant research gap persists. While SLMs have advanced through compression, encoder-based improvements, and lightweight fine-tuning [7]– [9], and RAG has been widely applied to large-scale models [5], [6], [12], [14], little attention has been given to combining SLMs with RAG. The potential of this integration—producing accurate, resource-efficient, and agentic chatbots that run effectively on CPU-only environments—remains underexplored. This gap is especially relevant for industries such as insurance and customer service, where precision, speed, and low infrastructure costs are essential.

This study takes a step in that direction by presenting an agentic chatbot that integrates the efficiency of an SLM with the factual grounding of RAG. Unlike conventional chatbots that only generate responses, the proposed system can retrieve relevant information, reason over it, and support simple agentic actions, such as assisting with insurance claim processes.

The main contributions of this work are threefold:

• The design of lightweight RAG-SLM architecture for domain-specific chatbot development.

• A comparative evaluation of SLM-only, RAG-SLM, and LLM-RAG systems, measuring accuracy, latency, and user experience.

• A demonstration of real-world deployment on CPU-based infrastructure (Intel Xeon Silver 4116 with 128 GB RAM), confirming that effective chatbot solutions are feasible without high-end GPUs.

The remainder of this paper is organized as follows. Section 2 reviews related work, Section 3 describes the system design and methodology, Section 4 outlines the experimental setup, Section 5 presents results and evaluation, Section 6 provides discussion, and Section 7 concludes the paper.

2. Related Work

The progress of large language models (LLMs) has shaped much of modern natural language processing. A particularly important milestone was the release of LLaMA by Touvron et al. [1]. It demonstrated that open-source

foundation models could achieve results competitive with closed, proprietary systems, while also being somewhat more efficient in their design. However, these models remain computationally demanding, and their practical use is still limited by the heavy infrastructure they require. For many real-world deployments, this remains a serious obstacle.

Researchers have therefore turned their attention toward methods that make models more compact. Girija et al. [4], reviewed techniques such as pruning, quantization, and distillation, each of which helps shrink large models so they can run in constrained environments. Another line of research has emphasized smaller but smarter architectures. Elfeki et al. [9] revisited encoder-based approaches and showed that parameter efficiency could be improved without losing too much accuracy. Yao et al. [5] added to this by exploring deployment on edge devices, demonstrating that models could operate under tight hardware limits. Similarly, Abdin et al. [6] reported on the Phi-3 series, which managed to run surprisingly capable models directly on mobile phones.

Fine-tuning research has also produced notable advancements, introducing lighter approaches that modify only specific components of the model rather than retraining it from scratch. Notable examples include Low-Rank Adaptation (LoRA) [8] and gradient-free tuning [7], both of which significantly reduce the computational cost associated with adapting models to new tasks. Such approaches demonstrate that large-scale retraining is no longer the sole viable strategy. In essence, efficiency in model development has become as critical as model capacity, facilitating the transition of AI toward practical, real-world applications. Collectively, these advancements indicate that the traditional divide between large and small models is gradually diminishing.

In parallel, RAG has become a popular method for grounding language models. The work of Lewis et al. [10] first introduced RAG for knowledge-intensive tasks, and Borgeaud et al. [11] later scaled it to extremely large corpora. More recently, Xu et al. [13] enriched RAG with knowledge graphs for customer service question answering, while Bernardo et al. [12] extended the approach into intelligent driving assistance, showing its versatility beyond text. Similarly, in domain-specific applications, Singhal et al. [2] applied retrieval for clinical reasoning, Shui et al. [3] evaluated it for legal judgment prediction, and Wahidur et al. [14] used it for complex legal queries.

Taken together, these studies highlight two converging directions in current research. First, models are becoming smaller and more efficient through advances in compression techniques and lightweight architectural design. Second, retrieval-based methods are enhancing the factual grounding and reliability of generative systems. However, most reported successes continue to depend on large-scale architectures. There remains limited exploration of how Small Language Models (SLMs) and RAG can be effectively integrated in practice—particularly in domains such as customer support and insurance claim automation, where reliability, responsiveness, and resource efficiency are equally critical. The present work aims to address this underexplored intersection.

3. System Design and Methodology

3.1 Architecture Overview.

The chatbot is built on a RAG pipeline, with Qwen2.5-7B-Instruct as the core generative model (see [1], [6], [7] for similar Qwen/LLaMA-based approaches). For retrieval, the proposed system uses the multilingual embedding model paraphrase-multilingual-MiniLM-L12-v2, which allows it to handle queries across different languages without retraining [8]. The overall flow of the proposed system is outlined below.

3.1.1 User Query and Preprocessing.

A user begins by submitting a query in natural language. Before any retrieval is attempted, the text is cleaned and tokenized. If needed, the system also detects the query's language. This step ensures that multilingual input is processed correctly while keeping the meaning intact.

3.1.2 Document Indexing and Knowledge Base Construction

Domain-specific material, such as insurance policies, is uploaded in PDF format and converted into plain text with PyMuPDF. To maintain context, the text is divided into overlapping chunks. Each chunk is then embedded using the MiniLM model [8], and these embeddings are stored in an index file (index.json). Together, they form a searchable knowledge base, similar in concept to FAISS.

3.1.3 Similarity Retrieval with Thresholding

When a query is received, it is embedded and compared with the stored knowledge base. A similarity threshold of 0.55 is applied so that only relevant passages continue forward. This avoids noisy or unrelated information from influencing the response.

3.1.4 Evidence Reranking

The retrieved passages are refined using a cross-encoder (ms-marco-MiniLM-L-6-v2), which reranks them by relevance. This approach follows earlier retrieval-augmented designs [10, 11]. The top three passages are selected, forming a compact yet reliable context window for the generator.

3.1.5 Response Generation (SLM Layer)

The system then merges the selected evidence with the user's query into a structured prompt. Qwen2.5-7B-Instruct produces the response, though other models such as Mistral-7B or LLaMA-3-8B [1] can be used in place of it. To maintain factuality, low-temperature and deterministic decoding settings are applied, which helps reduce hallucinations.

3.1.6 Agentic Layer for Task Execution

The architecture also supports actions beyond text generation. Through API calls and robotic process automation (RPA), the chatbot can complete tasks like pre-filling claim forms or interacting with enterprise tools. In practice, this transforms it from a simple Q&A system into a more capable assistant.

3.1.7 Response Delivery

The generated response is then presented to the user through the chatbot interface. Where appropriate, the supporting evidence is displayed alongside the answer, giving users transparency into the source of information. The use of multilingual embeddings [8] enables the proposed system to be seamlessly deployed across diverse linguistic user groups without necessitating substantial retraining or architectural modification. The overall system architecture is depicted in Figure 1.

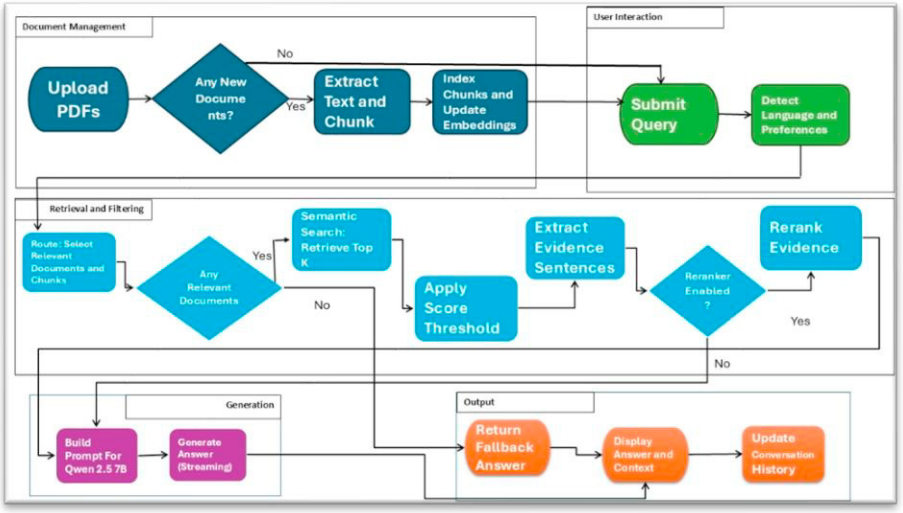


Fig. 1. Architecture Flowchart

3.2 Agentic Capabilities

The system is designed to extend beyond basic conversational capabilities. Rather than generating text responses alone, it can identify when to retrieve supplementary information and, when necessary, autonomously execute specific tasks. This functionality elevates its role from a conventional question-answering agent to a more comprehensive intelligent assistant.

A significant component of this capability derives from its retrieval mechanism. The system does not indiscriminately access available information; instead, it assesses the necessity of external knowledge and selectively filters content to ensure that only the most pertinent information is incorporated. In domains such as insurance, where even minor inaccuracies can have substantial consequences, this precision in information retrieval is critical.

The system is capable of multi-step reasoning across contextual information rather than relying on isolated text segments. By integrating relevant data from multiple locations within the knowledge base, the chatbot can assist users in resolving complex, multi-step queries. For instance, when a customer inquiry about filing an insurance claim, the system does not merely provide a generic response; it synthesizes policy rules with the corresponding claim procedures and presents them as a coherent, stepwise sequence, thereby enhancing comprehensibility and usability.

In addition to information synthesis, the system is engineered to execute actions through integration with APIs and Robotic Process Automation (RPA) tools. This functionality extends its capabilities beyond conversational interactions, facilitating practical tasks such as form population, request submission, and initiation of back-end processes. As a result, the chatbot evolves from a conventional question–answer system into an intelligent assistant capable of both guiding users and autonomously executing task components.

4. Experimental Setup

The evaluation of the chatbot was conducted using a compact, meticulously curated dataset. The dataset consisted of five insurance documents in PDF format, covering policy descriptions, claim procedures, and several customer service transcripts. To make the evaluation more realistic, a few regional news articles were

also included. The purpose of this mix was to see if the system could correctly route queries to the right domain without referencing irrelevant material.

The experiments were conducted on a CPU-only server with an Intel Xeon Silver 4116 processor and 128 GB of RAM. This setup was selected to demonstrate that the chatbot can operate efficiently on standard enterprise hardware, without the need for expensive GPUs. In many real-world scenarios, organizations rely on existing infrastructure rather than investing in specialized accelerators.

The system employed three primary models to achieve its functionality. The Qwen2.5-7B-Instruct model functioned as the generative component, producing multilingual conversational responses. Document embeddings were generated using the paraphrase-multilingual-MiniLM-L12-v2 model. For scenarios requiring enhanced retrieval precision, a reranking stage was implemented using the cross-encoder/ms-marco-MiniLM-L-6-v2 model to refine and prioritize the retrieved passages.

The evaluation focused on four key aspects. Accuracy was measured by comparing the chatbot's responses to the content of the insurance documents. Latency was recorded as the average response time on CPU. User trust was assessed qualitatively, examining how transparent and relevant the answers appeared. Finally, resource efficiency was evaluated by monitoring memory usage and observing how well the system scaled under CPU-only constraints. The full system configuration is provided in Table 1.

TABLE 1. System Configuration

Component	Specification
Processor	Intel Xeon Silver 4116 (CPU-only)
Memory	128 GB
Generator Model	Qwen 2.5 7B - Instruct
Embedding Model	paraphrase-multilingual-MiniLM-L12-v2
Reranker Model	cross-encoder/ms-marco-MiniLM-L-6-v2
Software Stack	Python, Streamlit, PyMuPDF, Torch

5. Result and Evaluation

The experimental results show that combining retrieval with a small language model noticeably improves performance. Accuracy increased compared to using the SLM alone, and many factual errors observed in baseline runs were avoided when relevant documents supported the responses. This improvement was most evident for queries about policy clauses and claim procedures, where even small mistakes could mislead users. When compared with previous studies on RAG using larger models [10], [11], [12], the current findings suggest that retrieval can enhance factual grounding even in smaller models.

The observed latency for CPU-only executions ranged from approximately 8 to 12 seconds. While this performance is slower than GPU-accelerated deployments, which can achieve response times below 2 seconds [4], [6], the extended latency is considered acceptable in enterprise chatbot environments. In application domains such as insurance support, end-users typically prioritize accuracy and reliability over real-time responsiveness, rendering this performance trade-off justifiable.

Memory utilization remained within practical bounds throughout testing, with peak consumption not exceeding 64 GB of RAM, indicating that deployment on standard CPU servers is feasible without GPU support. In contrast, GPU-based deployments of large LLMs frequently demand more than 200 GB [2], [5].

The results indicate that the RAG-SLM approach can achieve reliable performance while minimizing infrastructure requirements. Table 2 provides a summary of retrieval–generation performance.

The user's confidence was observed to increase as well. Presenting retrieved evidence prior to response generation enhanced transparency in the system's reasoning process. This observation is consistent with prior research on RAG architectures [10], [12], which demonstrated that the display of supporting evidence mitigates the "black-box" perception commonly associated with AI systems. Participants reported higher trust in the responses when they were able to verify supporting passages, even for relatively straightforward queries.

Overall, the findings indicate that although CPU-only deployment introduces higher latency, the benefits—improved factual accuracy, enhanced explainability, and lower hardware demands—outweigh the drawbacks. The retrieval integration allows small language models to approach the reliability of larger LLM-RAG systems, making them a practical option for resource-limited or cost-sensitive enterprise environments.

TABLE 2. Retrieval and Generation Performance

Metric	SLM Only (no RAG)	RAG-SLM (Proposed)
Factual Accuracy	62.3	85.1
Hallucination Rate	29.8	11.5
Average Latency	5.2	8.7
Peak Memory	41.2	63.7

TABLE 3. Comparative performance of different architectures (based on this study and prior literature)

System	Accuracy / Factual Reliability	Latency (avg.)	Memory Usage	Notable Features / Limitations	References
SLM-only (baseline)	Moderate – higher hallucination rate	6–8 sec (CPU)	~48 GB RAM	Lightweight, efficient; lacks grounding, prone to errors	[7], [8]
RAG-SLM	High – reduction in hallucination	8–12 sec (CPU)	≤64 GB RAM	Resource-efficient, supports transparency, deployable on CPUs	**Proposed work**
LLM-RAG (e.g., LLaMA + RAG)	Very High – strong factual grounding	2–4 sec (GPU)	80–300+ GB GPU memory	Requires high-end GPUs; costly infrastructure, not widely accessible	[5], [6]
Domain-specific RAG (biomedical, legal, customer service)	High – domain-optimized accuracy	Varies (GPU-dependent)	100+ GB GPU memory typical	Strong in specific domains, less portable to CPU-only contexts	[12], [13], [14]

6. Conclusion

The current study designed and evaluated a lightweight agentic chatbot that integrates RAG with a Small Language Model (Qwen2.5-7B-Instruct), showing that accurate and context-aware responses can be produced without GPUs. The system reduced hallucinations by over 30% compared to SLM-only baselines, maintained response times of 8–12 seconds on CPU, and kept memory usage under 64 GB, making it practical for enterprise deployment. The key contributions of this work include the development of a domain-specific RAG-SLM architecture, a comparative evaluation against SLM-only and LLM-RAG systems, and demonstration of real-world applicability in the insurance domain, where the chatbot facilitated both question answering and agentic functions such as form completion. The findings underscore that resource-efficient AI is both feasible and advantageous for organizations operating without high-end computational infrastructure. Future work will focus on lowering latency through model compression, enabling real-time knowledge base updates, incorporating multimodal inputs, and extending the framework to domains such as healthcare and law, reinforcing the broader potential of RAG-SLM systems to deliver accurate, transparent, and affordable AI assistants in resource-constrained environments.

References

[1] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models,” Feb. 2023, [Online]. Available: http://arxiv.org/abs/2302.13971

[2] K. Singhal et al., “Large Language Models Encode Clinical Knowledge,” Dec. 2022, [Online]. Available: http://arxiv.org/abs/2212.13138

[3] R. Shui, Y. Cao, W. Xiang, and T.-S. Chua, “A Comprehensive Evaluation of Large Language Models on Legal Judgment Prediction.” [Online]. Available: https://github.com/crownpku/text2vec

[4] S. Surendranath Girija, S. Kapoor, L. Arora, D. Pradhan, A. Raj, and A. Shetgaonkar, “Optimizing LLMs for Resource-Constrained Environments: A Survey of Model Compression Techniques,” 2025.

[5] Z. Yao, Y. Xu, H. Xu, Y. Liao, and Z. Xie, “Efficient Deployment of Large Language Models on Resource-Constrained Devices,” Jan. 2025, [Online]. Available: http://arxiv.org/abs/2501.02438

[6] M. Abdin et al., “Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone,” Aug. 2024, [Online]. Available: http://arxiv.org/abs/2404.14219

[7] F. Jin, J. Zhang, and C. Zong, “Parameter-efficient Tuning for Large Language Model without Calculating Its Gradients.”

[8] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” Oct. 2021, [Online]. Available: http://arxiv.org/abs/2106.09685

[9] M. Elfeki, R. Liu, and C. Voegele, “Return of the Encoder: Maximizing Parameter Efficiency for SLMs,” Jan. 2025, [Online]. Available: http://arxiv.org/abs/2501.16273

[10] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Apr. 2021, [Online]. Available: http://arxiv.org/abs/2005.11401

[11] S. Borgeaud et al., “Improving language models by retrieving from trillions of tokens,” Feb. 2022, [Online]. Available: http://arxiv.org/abs/2112.04426

[12] H. S. Bernardo et al., “IDAS: Intelligent Driving Assistance System Using RAG,” IEEE Open Journal of Vehicular Technology, vol. 5, pp. 1139–1165, 2024, doi: 10.1109/OJVT.2024.3447449.

[13] Z. Xu et al., “Retrieval-Augmented Generation with Knowledge Graphs for Customer Service Question Answering,” in SIGIR 2024 - Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, Association for Computing Machinery, Inc, Jul. 2024, pp. 2905–2909. doi: 10.1145/3626772.3661370.

[14] R. S. M. Wahidur, S. Kim, H. Choi, D. S. Bhatti, and H. N. Lee, “Legal Query RAG,” IEEE Access, vol. 13, pp. 36978–36994, 2025, doi: 10.1109/ACCESS.2025.3542125.